SLP-NLP Lab 2

Επεξεργασία Φωνής και Φυσικής Γλώσσας
Εαρινό Εξάμηνο 2025-26

Μαντζώρος Γεράσιμος, Παπαδημητρίου Νίκη

Απρίλιος 2026

Περιεχόμενα

Μέρος Α΄: Προπαρασκευή **2**
Ζητούμενο 1 – Κωδικοποίηση Επισημειώσεων . . . 2
Ζητούμενο 2 – Λεκτική Ανάλυση (Tokenization) . . . 2
Ζητούμενο 3 – Κωδικοποίηση Παραδειγμάτων . . . 3
Ζητούμενο 4 – Embedding Layer . . . 5
Ζητούμενο 5 – Output Layer(s) . . . 5
Ζητούμενο 6 – Forward Pass . . . 6
Ζητούμενο 7 – DataLoaders . . . 7
Ζητούμενο 8 – Βελτιστοποίηση . . . 8
Ζητούμενο 9 – Εκπαίδευση . . . 10
Ζητούμενο 10 – Αξιολόγηση Baseline Μοντέλου . . . 11
Ζητούμενο 11 – Κατηγοριοποίηση με Χρήση LLM . . . 12

Μέρος Β΄: Κύρια Άσκηση **13**
Ερώτημα 1 – Mean + Max Pooling . . . 13
Ερώτημα 2 – LSTM . . . 14
Ερώτημα 3 – Self-Attention . . . 15
Ερώτημα 4 – Multi-Head Attention . . . 17
Ερώτημα 5 – Transformer-Encoder . . . 18
Ερώτημα 6 – Pre-Trained Transformers . . . 19
Ερώτημα 7 – Fine-Tuning Pre-Trained Transformers . . . 20
Ερώτημα 8 – Διάλογος με ChatGPT . . . 21

Συγκεντρωτικά Αποτελέσματα **22**

Μέρος Α΄: Προπαρασκευή

Για τα ζητούμενα της προπαρασκευής χρησιμοποιήσαμε το **Sentence Polarity Dataset (MR)**.

Ζητούμενο 1 – Κωδικοποίηση Επισημειώσεων

Για την κωδικοποίηση των επισημειώσεων χρησιμοποιήσαμε την κλάση LabelEncoder από τη βιβλιοθήκη scikit-learn. Αρχικά, τα labels των παραδειγμάτων εκπαίδευσης βρίσκονταν σε μορφή κειμένου, όπως positive, negative και neutral. Ο LabelEncoder εκπαιδεύτηκε πάνω στα labels του training set και στη συνέχεια χρησιμοποιήθηκε για να μετατρέψει τόσο τα labels του training set όσο και του test set σε ακέραιες τιμές, έτσι ώστε η αντιστοίχιση να είναι κοινή και συνεπής και στα δύο σύνολα.

Στην υλοποίηση, αρχικά αποθηκεύσαμε τα πρώτα 10 αρχικά labels του training set, έπειτα εφαρμόσαμε το fit του LabelEncoder στα labels του training set και στη συνέχεια χρησιμοποιήσαμε τη μέθοδο transform για την αριθμητική κωδικοποίησή τους. Με αυτόν τον τρόπο, κάθε κλάση αντιστοιχίζεται σε έναν μοναδικό ακέραιο αριθμό, μορφή απαραίτητη για την εκπαίδευση του νευρωνικού δικτύου.

Πίνακας 1: Έξοδος Ζητούμενου 1(main.py:EX1)

Label	Encoded value
positive	1
positive	1
positive	1
positive	1
positive	1
positive	1
positive	1
positive	1
positive	1
positive	1

Ζητούμενο 2 – Λεκτική Ανάλυση (Tokenization)

Για τη λεκτική ανάλυση χρησιμοποιήσαμε την κλάση SentenceDataset στο αρχείο dataloading.py. Στην αρχικοποίηση του Dataset, κάθε πρόταση του MR μετατρέπεται σε ακολουθία από tokens με χρήση της εντολής sentence.lower().split(), δηλαδή το κείμενο μετατρέπεται πρώτα σε πεζά και στη συνέχεια χωρίζεται με βάση τα κενά. Τα επεξεργασμένα παραδείγματα αποθηκεύονται στη μεταβλητή self.data της κλάσης, ώστε να είναι διαθέσιμα στα επόμενα στάδια της προεπεξεργασίας.

Τα πρώτα 10 tokenized παραδείγματα του training set φαίνονται παρακάτω.

```
[0] tokens = ['the', 'rock', 'is', 'destined', 'to', 'be', 'the', '21st',
              "century's", 'new', '"', 'conan', '"', 'and', 'that', "he's",
              'going', 'to', 'make', 'a', 'splash', 'even', 'greater', 'than',
              'arnold', 'schwarzenegger', ',', 'jean-claud', 'van', 'damme',
              'or', 'steven', 'segal', '.']

[1] tokens = ['the', 'gorgeously', 'elaborate', 'continuation', 'of', '"',
              'the', 'lord', 'of', 'the', 'rings', '"', 'trilogy', 'is', 'so',
              'huge', 'that', 'a', 'column', 'of', 'words', 'cannot', 'adequately',
              'describe', 'co-writer/director', 'peter', "jackson's", 'expanded',
```

```
            'vision', 'of', 'j', '.', 'r', '.', 'r', '.', "tolkien's",
            'middle-earth', '.']

[2] tokens = ['effective', 'but', 'too-tepid', 'biopic']

[3] tokens = ['if', 'you', 'sometimes', 'like', 'to', 'go', 'to', 'the',
            'movies', 'to', 'have', 'fun', ',', 'wasabi', 'is', 'a', 'good',
            'place', 'to', 'start', '.']

[4] tokens = ['emerges', 'as', 'something', 'rare', ',', 'an', 'issue',
            'movie', "that's", 'so', 'honest', 'and', 'keenly', 'observed',
            'that', 'it', "doesn't", 'feel', 'like', 'one', '.']

[5] tokens = ['the', 'film', 'provides', 'some', 'great', 'insight', 'into',
            'the', 'neurotic', 'mindset', 'of', 'all', 'comics', '--', 'even',
            'those', 'who', 'have', 'reached', 'the', 'absolute', 'top', 'of',
            'the', 'game', '.']

[6] tokens = ['offers', 'that', 'rare', 'combination', 'of', 'entertainment',
            'and', 'education', '.']

[7] tokens = ['perhaps', 'no', 'picture', 'ever', 'made', 'has', 'more',
            'literally', 'showed', 'that', 'the', 'road', 'to', 'hell', 'is',
            'paved', 'with', 'good', 'intentions', '.']

[8] tokens = ['steers', 'turns', 'in', 'a', 'snappy', 'screenplay', 'that',
            'curls', 'at', 'the', 'edges', ';', "it's", 'so', 'clever', 'you',
            'want', 'to', 'hate', 'it', '.', 'but', 'he', 'somehow', 'pulls',
            'it', 'off', '.']

[9] tokens=['take', 'care', 'of', 'my', 'cat', 'offers', 'a',
           'refreshingly', 'different', 'slice', 'of', 'asian',
           'cinema', '.']
```

Ζητούμενο 3 – Κωδικοποίηση Παραδειγμάτων (Λέξεων)

Στο τελικό στάδιο της προεπεξεργασίας υλοποιήσαμε τη μέθοδο __getitem__ της κλάσης SentenceDataset, ώστε κάθε πρόταση να επιστρέφεται σε μορφή κατάλληλη για την εκπαίδευση του νευρωνικού δικτύου. Πιο συγκεκριμένα, κάθε token αντιστοιχίζεται σε έναν ακέραιο δείκτη μέσω του λεξικού word2idx, το οποίο προκύπτει από τη φόρτωση των προεκπαιδευμένων word embeddings. Αν κάποιος όρος δεν υπάρχει στο λεξικό, αντιστοιχίζεται στο ειδικό token <unk>.

Στη συνέχεια, όλες οι προτάσεις προσαρμόζονται σε κοινό μέγιστο μήκος. Για το MR χρησιμοποιήθηκε μήκος 35, όπως ορίζεται στο dataloading.py. Αν μια πρόταση είναι μικρότερη από αυτό το μήκος, συμπληρώνεται με μηδενικά στο τέλος (zero-padding), ενώ αν είναι μεγαλύτερη, αποκόπτεται. Με αυτόν τον τρόπο κάθε παράδειγμα έχει σταθερές διαστάσεις και μπορεί να οργανωθεί σε mini-batches.

Η μέθοδος __getitem__ επιστρέφει τρία στοιχεία:

- την κωδικοποιημένη πρόταση ως διάνυσμα ακέραιων δεικτών,
- το κωδικοποιημένο label,

- το πραγματικό μήκος της πρότασης πριν από το padding.

Τα πρώτα 5 παραδείγματα στην αρχική τους μορφή και στη μορφή που τα επιστρέφει η κλάση SentenceDataset έχουν την ακόλουθη δομή.

```
Original : the rock is destined to be the 21st century's new " conan " and
           that he's going to make a splash even greater than arnold
           schwarzenegger , jean-claud van damme or steven segal .

Encoded  : [     1,   1138,     15,  10454,      5,     31,      1,
              5034, 400001,     51,      9,  18513,      9,      6,
                13, 400001,    223,      5,    160,      8,  16807,
               152,   1414,     74,   5819,   6681,      2, 400001,
              1462,  43708,     47,   4412,  26985,      3,      0]

Original : the gorgeously elaborate continuation of " the lord of the rings "
           trilogy is so huge that a column of words cannot adequately
           describe co-writer/director peter jackson's expanded vision of
           j . r . r . tolkien's middle-earth .

Encoded  : [     1,  78616,   5135,  10117,      4,      9,      1,
              2371,      4,      1,   6820,      9,  12305,     15,
               101,   1325,     13,      8,   3236,      4,   1375,
              1120,  12424,   4467, 400001,   1295, 400001,   2853,
              3139,      4,   6892,      3,   1912,      3,   1912]

Original : effective but too-tepid biopic

Encoded  : [   2038,     35, 400001,  34277,      0,      0,      0,
                  0,      0,      0,      0,      0,      0,      0,
                  0,      0,      0,      0,      0,      0,      0,
                  0,      0,      0,      0,      0,      0,      0,
                  0,      0,      0,      0,      0,      0,      0]

Original : if you sometimes like to go to the movies to have fun , wasabi
           is a good place to start .

Encoded  : [    84,     82,   1072,    118,      5,    243,      5,
                 1,   2460,      5,     34,   2906,      2,  66408,
                15,      8,    220,    242,      5,    466,      3,
                 0,      0,      0,      0,      0,      0,      0,
                 0,      0,      0,      0,      0,      0,      0]

Original : emerges as something rare , an issue movie that's so honest and
           keenly observed that it doesn't feel like one .

Encoded  : [ 12398,     20,    646,   2349,      2,     30,    496,
              1006, 400001,    101,   6082,      6,  23499,   4583,
                13,     21, 400001,    999,    118,     49,      3,
                 0,      0,      0,      0,      0,      0,      0,
                 0,      0,      0,      0,      0,      0,      0]
```

Ζητούμενο 4 – Embedding Layer

Ορίσαμε self.embedding = nn.Embedding.from_pretrained(torch.FloatTensor(embeddings)) για αρχικοποίηση από τα προ-εκπαιδευμένα διανύσματα και θέσαμε self.embedding.weight.requires_grad = trainable_emb για έλεγχο fine-tuning.

Γιατί αρχικοποιούμε το embedding layer με τα προ-εκπαιδευμένα word embeddings· Αρχικοποιούμε το embedding layer με προεκπαιδευμένα word embeddings, επειδή αυτά έχουν ήδη μάθει στατιστικές και σημασιολογικές σχέσεις μεταξύ των λέξεων από πολύ μεγάλα σώματα κειμένου. Έτσι, το μοντέλο ξεκινά από μία πολύ πιο πλούσια και ενημερωμένη αναπαράσταση του λεξιλογίου, αντί να χρειάζεται να μάθει από την αρχή αυτές τις σχέσεις μόνο από το σχετικά περιορισμένο dataset της άσκησης. Αυτό συνήθως οδηγεί σε ταχύτερη σύγκλιση και καλύτερη γενίκευση.

Γιατί κρατάμε παγωμένα τα βάρη του embedding layer κατά την εκπαίδευση· Κρατάμε παγωμένα τα βάρη του embedding layer κατά την εκπαίδευση, ώστε να διατηρηθεί η πληροφορία που έχει ήδη κωδικοποιηθεί στα προεκπαιδευμένα embeddings. Αν τα βάρη ενημερώνονταν πάνω σε μικρό dataset, υπάρχει κίνδυνος να αλλοιωθούν οι αρχικές σημασιολογικές σχέσεις και το μοντέλο να υπερπροσαρμοστεί στα δεδομένα εκπαίδευσης. Επιπλέον, το πάγωμα του embedding layer μειώνει τον αριθμό των παραμέτρων που πρέπει να εκπαιδευτούν και άρα απλοποιεί τη διαδικασία βελτιστοποίησης.

Ζητούμενο 5 – Output Layer(s)

Ορίσαμε self.linear = nn.Linear(dim, hidden_size) και self.relu = nn.ReLU() ως μη γραμμικό μετασχηματισμό, και self.output = nn.Linear(hidden_size, output_size) ως τελικό layer ταξινόμησης.

Γιατί βάζουμε μη γραμμική συνάρτηση ενεργοποίησης στο προτελευταίο layer· Βάζουμε μία μη γραμμική συνάρτηση ενεργοποίησης στο προτελευταίο layer, επειδή χωρίς αυτήν το δίκτυο δεν θα μπορούσε να μάθει πραγματικά μη γραμμικούς μετασχηματισμούς της αναπαράστασης του κειμένου. Η μη γραμμικότητα επιτρέπει στο μοντέλο να αποτυπώνει πιο σύνθετες σχέσεις ανάμεσα στα χαρακτηριστικά της πρότασης και στις κλάσεις, κάτι ιδιαίτερα σημαντικό σε προβλήματα όπως η κατηγοριοποίηση συναισθήματος, όπου η απόφαση δεν προκύπτει συνήθως από έναν απλό γραμμικό διαχωρισμό.

Η μη γραμμικότητα τοποθετείται στο προτελευταίο layer, δηλαδή αμέσως μετά τον πρώτο γραμμικό μετασχηματισμό της αναπαράστασης του κειμένου, επειδή σε αυτό το σημείο το μοντέλο έχει ήδη συνοψίσει την πρόταση σε ένα ενιαίο διάνυσμα χαρακτηριστικών και θέλουμε να μάθει έναν πιο εκφραστικό μετασχηματισμό πριν από την τελική ταξινόμηση. Το τελευταίο layer, αντίθετα, πρέπει να παράγει τα τελικά logits για τις κλάσεις και γι΄ αυτό παραμένει γραμμικό. Αν εφαρμόζαμε εκεί μία επιπλέον μη γραμμικότητα, θα περιορίζαμε ή θα παραμορφώναμε τις τιμές των logits, κάτι που δεν είναι επιθυμητό όταν στη συνέχεια χρησιμοποιούνται συναρτήσεις κόστους όπως CrossEntropyLoss ή BCEWithLogitsLoss.

Τι διαφορά θα είχε αν είχαμε 2 ή περισσότερους γραμμικούς μετασχηματισμούς στη σειρά· Αν είχαμε δύο ή περισσότερους γραμμικούς μετασχηματισμούς στη σειρά, χωρίς καμία μη γραμμική συνάρτηση ανάμεσά τους, τότε το αποτέλεσμα θα ήταν ισοδύναμο με έναν μόνο γραμμικό μετασχηματισμό. Πράγματι, αν

$$h = W_1 x + b_1$$

και

$$y = W_2 h + b_2,$$

τότε προκύπτει

$$y = W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2).$$

Αν θέσουμε

$$W' = W_2 W_1 \qquad \text{και} \qquad b' = W_2 b_1 + b_2,$$

τότε

$$y = W'x + b'.$$

Άρα, η σύνθεση δύο γραμμικών layers παραμένει γραμμική. Το ίδιο ισχύει και για περισσότερα γραμμικά layers: όλα μπορούν να συγχωνευθούν σε έναν μόνο γραμμικό μετασχηματισμό. Επομένως, χωρίς μη γραμμική ενεργοποίηση, το μοντέλο παραμένει ουσιαστικά ένας γραμμικός ταξινομητής και δεν αυξάνει την εκφραστική του ικανότητα.

Ζητούμενο 6 – Forward Pass

Στο forward pass του μοντέλου, η είσοδος είναι ένα mini-batch προτάσεων με διαστάσεις (B, T), όπου B είναι το batch size και T το μέγιστο μήκος πρότασης. Αρχικά, κάθε ακέραιος δείκτης λέξης προβάλλεται μέσω του embedding layer στο αντίστοιχο word embedding, οπότε η είσοδος μετατρέπεται από σχήμα (B, T) σε (B, T, d), όπου d είναι η διάσταση των embeddings. Έπειτα, για κάθε πρόταση υπολογίζεται μία ενιαία αναπαράσταση με mean pooling, δηλαδή με τον μέσο όρο των word embeddings της πρότασης. Επειδή οι προτάσεις έχουν διαφορετικά πραγματικά μήκη, για τον σωστό υπολογισμό του μέσου όρου χρησιμοποιούνται τα lengths, ώστε να εξαιρούνται τα zero-padded timesteps. Αν συμβολίσουμε με $e_1, e_2, \ldots, e_N$ τα embeddings των πραγματικών λέξεων μιας πρότασης, τότε η αναπαράστασή της δίνεται από:

$$u = \frac{1}{N} \sum_{i=1}^{N} e_i$$

όπου N είναι το πραγματικό μήκος της πρότασης και όχι το συνολικό padded μήκος. Στη συνέχεια, η αναπαράσταση u μετασχηματίζεται από το γραμμικό layer και τη μη γραμμική ενεργοποίηση ReLU, ώστε να παραχθεί μία νέα λανθάνουσα αναπαράσταση, και τέλος αυτή προβάλλεται από το output layer στον χώρο των κλάσεων, παράγοντας τα τελικά logits.

Διαισθητική ερμηνεία της αναπαράστασης κέντρο-βάρους: Αν θεωρήσουμε ότι κάθε διάσταση του embedding space αντιστοιχεί σε μία αφηρημένη σημασιολογική ή συντακτική ιδιότητα, τότε η αναπαράσταση που κατασκευάζουμε με mean pooling μπορεί να ερμηνευθεί διαισθητικά ως το κέντρο βάρους της πρότασης μέσα στον χώρο των embeddings. Η αναλογία με το κέντρο βάρους προκύπτει επειδή η τελική αναπαράσταση είναι ο αριθμητικός μέσος των επιμέρους διανυσμάτων:

$$u = \frac{e_1 + e_2 + \cdots + e_N}{N}$$

που είναι ακριβώς ο τύπος του κέντρου μάζας ενός συνόλου σημείων με ίσα βάρη. Επομένως, το διάνυσμα αυτό αποτυπώνει τη μέση θέση της πρότασης στον σημασιολογικό χώρο, δηλαδή μία συνολική περίληψη του περιεχομένου της. Με άλλα λόγια, το μοντέλο αγνοεί τις επιμέρους διακυμάνσεις και κρατά μια μέση 'κατεύθυνση νοήματος' της πρότασης.

Πιθανές αδυναμίες της προσέγγισης για αναπαράσταση κειμένων: Η συγκεκριμένη προσέγγιση, παρότι απλή και υπολογιστικά αποδοτική, έχει σημαντικές αδυναμίες ως αναπαράσταση κειμένου. Πρώτον, αγνοεί πλήρως τη σειρά των λέξεων, αφού ο μέσος όρος των embeddings παραμένει ο ίδιος ανεξάρτητα από τη διάταξή τους. Άρα προτάσεις με ίδιες λέξεις αλλά διαφορετική σειρά μπορεί να έχουν πολύ παρόμοια ή και ίδια αναπαράσταση. Δεύτερον, δεν λαμβάνει υπόψη αλληλεπιδράσεις μεταξύ λέξεων, όπως άρνηση, ειρωνεία ή εξαρτήσεις μεγάλης απόστασης. Τρίτον, όλες οι λέξεις συνεισφέρουν ισότιμα στον μέσο όρο, παρότι στην πράξη ορισμένες λέξεις είναι πολύ πιο σημαντικές από άλλες για την κατηγορία του συναισθήματος. Τέλος, με τη διαδικασία του μέσου όρου χάνονται τοπικές και έντονες ενδείξεις, αφού η πληροφορία εξομαλύνεται σε ένα μόνο διάνυσμα σταθερής διάστασης.

Ζητούμενο 7 – DataLoaders

Ορίσαμε train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True) και test_loader = DataLoader(test_set, batch_size=BATCH_SIZE, shuffle=False).

Συνέπειες μικρών και μεγάλων mini-batches στην εκπαίδευση: Το μέγεθος του mini-batch επηρεάζει σημαντικά τη διαδικασία εκπαίδευσης. Μικρά mini-batches οδηγούν σε πιο θορυβώδεις εκτιμήσεις της κλίσης του σφάλματος, επειδή κάθε βήμα ενημέρωσης βασίζεται σε λιγότερα παραδείγματα. Αυτό μπορεί να κάνει την εκπαίδευση λιγότερο σταθερή, αλλά πολλές φορές βοηθά το μοντέλο να εξερευνήσει καλύτερα τον χώρο των λύσεων και να αποφύγει φτωχά τοπικά ελάχιστα. Στο ακραίο όριο, ένα mini-batch μεγέθους 1 σημαίνει ότι η ενημέρωση των βαρών βασίζεται ουσιαστικά σε ένα μόνο σημείο του dataset κάθε φορά. Σε αυτήν την περίπτωση το μοντέλο προσπαθεί σε κάθε βήμα να προσαρμοστεί σε μία μόνο παρατήρηση, γεγονός που οδηγεί σε πολύ αργή και θορυβώδη εξερεύνηση του χώρου βελτιστοποίησης.

Αντίθετα, μεγάλα mini-batches παρέχουν πιο σταθερές και ακριβείς εκτιμήσεις της κλίσης, επειδή κάθε ενημέρωση προκύπτει από τον μέσο όρο περισσότερων παραδειγμάτων. Αν για ένα mini-batch B το συνολικό κόστος γράφεται ως

$$L_B = \frac{1}{|B|} \sum_{i \in B} L_i,$$

τότε η αντίστοιχη κλίση είναι

$$\nabla L_B = \frac{1}{|B|} \sum_{i \in B} \nabla L_i.$$

Όσο μεγαλώνει το $|B|$, τόσο πιο αντιπροσωπευτική γίνεται αυτή η μέση κλίση του συνολικού dataset. Ωστόσο, τα πολύ μεγάλα mini-batches απαιτούν περισσότερη μνήμη, αυξάνουν το υπολογιστικό κόστος κάθε βήματος και μπορεί να οδηγήσουν σε λιγότερο ευέλικτη εξερεύνηση του χώρου των λύσεων. Επομένως, στην πράξη επιλέγεται ένας ενδιάμεσος συμβιβασμός ανάμεσα στη σταθερότητα και στην ταχύτητα εκπαίδευσης.

Γιατί ανακατεύουμε τη σειρά των mini-batches σε κάθε εποχή· Η ανακάτεμα (shuffle) της σειράς των mini-batches σε κάθε εποχή είναι επίσης σημαντικό. Αν τα δεδομένα δίνονταν πάντα με την ίδια σειρά, το μοντέλο θα έβλεπε κάθε φορά την ίδια ακολουθία παραδειγμάτων και οι ενημερώσεις των βαρών θα μπορούσαν να επηρεάζονται συστηματικά από τη διάταξη του dataset. Αυτό έχει σημασία επειδή κάθε gradient update δεν επηρεάζει μόνο το μέτρο της μεταβολής των βαρών, αλλά και την κατεύθυνση προς την οποία το μοντέλο μετακινείται στον χώρο παραμέτρων. Αν η σειρά των παραδειγμάτων είναι μεροληπτική, τότε και οι διαδοχικές κατευθύνσεις των ενημερώσεων γίνονται μεροληπτικές.

Για παράδειγμα, αν πολλά παραδείγματα της ίδιας κλάσης εμφανίζονταν συνεχόμενα, το μοντέλο θα έκανε για αρκετά βήματα ενημερώσεις προς παρόμοια κατεύθυνση, πριν προλάβει να δει επαρκώς

παραδείγματα από άλλες κλάσεις. Αυτό μπορεί να οδηγήσει σε ταλαντώσεις, πιο αργή σύγκλιση ή προσωρινή υπερπροσαρμογή σε συγκεκριμένα μοτίβα της σειράς των δεδομένων. Με το shuffle, κάθε εποχή αντιστοιχεί σε διαφορετική δειγματοληψία της σειράς των παραδειγμάτων, οπότε οι κλίσεις που υπολογίζονται είναι λιγότερο μεροληπτικές ως προς τη σειρά και πιο αντιπροσωπευτικές της συνολικής κατανομής των δεδομένων. Έτσι η πορεία βελτιστοποίησης γίνεται πιο σταθερή και πιο αξιόπιστη.

Note: Παρότι το early stopping δεν αποτελεί μέρος της λειτουργίας των DataLoaders, η χρήση σωστού shuffle συμβάλλει έμμεσα και εκεί, επειδή μια πιο σταθερή και λιγότερο μεροληπτική εκπαίδευση οδηγεί συνήθως σε πιο ομαλή συμπεριφορά του validation loss. Έτσι, το κριτήριο του early stopping βασίζεται σε πιο αξιόπιστη εικόνα της πορείας εκπαίδευσης και της ικανότητας γενίκευσης του μοντέλου.

Ζητούμενο 8 – Βελτιστοποίηση

Στη βελτιστοποίηση του μοντέλου έπρεπε να οριστούν το κατάλληλο κριτήριο σφάλματος, οι παράμετροι που θα εκπαιδεύονται και ο αλγόριθμος βελτιστοποίησης.

Κριτήριο (Loss): Χρησιμοποιούμε BCEWithLogitsLoss όταν το πρόβλημα είναι δυαδικής ταξινόμησης, επειδή το μοντέλο επιστρέφει ένα μόνο logit ανά παράδειγμα. Όταν υπάρχουν περισσότερες από δύο κλάσεις, χρησιμοποιούμε CrossEntropyLoss, η οποία δέχεται ένα διάνυσμα logits μεγέθους ίσου με το πλήθος των κλάσεων. Έτσι, η επιλογή του loss συνδέεται άμεσα με τη μορφή του output layer.

Listing 1: Code for loss selection (main.py)

```
output_size = 1 if n_classes == 2 else n_classes

if n_classes == 2:
    criterion = torch.nn.BCEWithLogitsLoss()
else:
    criterion = torch.nn.CrossEntropyLoss()
```

Παράμετροι προς βελτιστοποίηση: Δεν θέλουμε να εκπαιδεύσουμε όλες τις παραμέτρους του δικτύου, αλλά μόνο εκείνες που έχουν οριστεί ως εκπαιδεύσιμες. Για αυτόν τον λόγο χρησιμοποιούμε φίλτρο πάνω στις παραμέτρους του μοντέλου και κρατάμε μόνο όσες έχουν requires_grad=True. Με αυτόν τον τρόπο, αν κάποιο τμήμα του μοντέλου, όπως το embedding layer, είναι παγωμένο, δεν ενημερώνεται κατά τη βελτιστοποίηση.

Listing 2: Code for parameter selection (main.py)

```
model = models[args.model]
model.to(DEVICE)

parameters = filter(lambda p: p.requires_grad, model.parameters())
```

Optimizer: Ως optimizer χρησιμοποιήσαμε τον Adam με learning rate 10^{-3}. Ο Adam είναι κατάλληλη επιλογή, επειδή προσαρμόζει δυναμικά το μέγεθος του βήματος για κάθε παράμετρο και συνήθως οδηγεί σε πιο σταθερή και γρήγορη εκπαίδευση από τον απλό stochastic gradient descent, ιδιαίτερα σε μικρά και μεσαία deep learning προβλήματα. Πιο συγκεκριμένα, αν g_t είναι η κλίση της συνάρτησης κόστους ως προς μία παράμετρο στη χρονική στιγμή t, ο Adam υπολογίζει έναν εκθετικά σταθμισμένο μέσο όρο της κλίσης και του τετραγώνου της:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

και στη συνέχεια χρησιμοποιεί τις διορθωμένες εκτιμήσεις

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

ώστε η ενημέρωση της παραμέτρου θ_t να δίνεται από:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

όπου η είναι το learning rate. Με αυτόν τον τρόπο, το βήμα ενημέρωσης προσαρμόζεται δυναμικά ανά παράμετρο, κάτι που βελτιώνει τη σταθερότητα και την ταχύτητα σύγκλισης.

Listing 3: Code for optimizer definition (main.py)

```
parameters = filter(lambda p: p.requires_grad, model.parameters())
optimizer = torch.optim.Adam(parameters, lr=1e-3)
```

Listing 4: Code for freezing the embedding layer (models.py)

```
self.embedding = nn.Embedding.from_pretrained(torch.FloatTensor(
    embeddings))
self.embedding.weight.requires_grad = trainable_emb
```

Υπερπαράμετροι:

- Learning rate: 10^{-3}
- Batch size: 64
- Εποχές: έως 50 (με early stopping, patience=5)
- Άλλες: hidden size=100

Οι βασικές υπερπαράμετροι που χρησιμοποιήθηκαν είναι learning rate 10^{-3}, batch size 64, μέγιστος αριθμός εποχών 50 και hidden size 100. Η επιλογή αυτών των τιμών δεν προέκυψε από εξαντλητική αναζήτηση υπερπαραμέτρων, αλλά βασίστηκε κυρίως στη δομή και στις αρχικές ρυθμίσεις του δοθέντος κώδικα της άσκησης, δηλαδή στις προτεινόμενες τιμές του εργαστηρίου, οι οποίες λειτουργούν ως ένα λογικό και σταθερό αρχικό σημείο για την εκπαίδευση.

Πιο συγκεκριμένα, το learning rate 10^{-3} επιλέχθηκε ως μία κλασική και ασφαλής τιμή για τον Adam, καθώς είναι συνήθως αρκετά μικρό ώστε να εξασφαλίζει σταθερή σύγκλιση, αλλά και αρκετά μεγάλο ώστε η εκπαίδευση να προχωρά με ικανοποιητική ταχύτητα. Το batch size 64 αποτελεί έναν πρακτικό συμβιβασμό ανάμεσα στη σταθερότητα της εκτίμησης της κλίσης και στο υπολογιστικό κόστος κάθε βήματος εκπαίδευσης. Ο μέγιστος αριθμός εποχών ορίστηκε σε 50, όχι επειδή αναμέναμε απαραίτητα να ολοκληρωθούν όλες, αλλά ώστε το μοντέλο να έχει επαρκές περιθώριο εκπαίδευσης, ενώ η πραγματική διάρκεια της εκπαίδευσης ελέγχεται από το early stopping με patience 5. Με αυτόν τον τρόπο αποφεύγεται τόσο η πρόωρη διακοπή όσο και η άσκοπη συνέχιση της εκπαίδευσης όταν το validation loss δεν βελτιώνεται πλέον.

Τέλος, το hidden size 100 χρησιμοποιήθηκε ως μία σχετικά μικρή αλλά επαρκής διάσταση για τον ενδιάμεσο λανθάνοντα χώρο, ώστε το μοντέλο να έχει αρκετή εκφραστική ικανότητα χωρίς να αυξάνεται υπερβολικά ο αριθμός των παραμέτρων. Συνολικά, οι τιμές αυτές επιλέχθηκαν ως πρακτικές και σταθερές ρυθμίσεις βάσης, συμβατές με το μέγεθος του προβλήματος και τις οδηγίες της άσκησης.

Ζητούμενο 9 – Εκπαίδευση

Στο στάδιο της εκπαίδευσης υλοποιούνται δύο βασικές συναρτήσεις: η train_dataset, η οποία εκτελεί ένα πλήρες πέρασμα εκπαίδευσης πάνω στα mini-batches μιας εποχής, και η eval_dataset, η οποία χρησιμοποιείται για αξιολόγηση χωρίς ενημέρωση βαρών. Στην εκπαίδευση, για κάθε batch μεταφέρονται τα inputs, τα labels και τα lengths στη σωστή συσκευή, μηδενίζονται οι προηγούμενες κλίσεις με optimizer.zero_grad(), εκτελείται το forward pass, υπολογίζεται το loss και στη συνέχεια εφαρμόζεται backpropagation. Αν η συνάρτηση κόστους συμβολιστεί ως $L(\theta)$, η βασική ενημέρωση των παραμέτρων ακολουθεί το σχήμα

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta),$$

όπου η είναι ο ρυθμός μάθησης. Στην πράξη, αυτή η διαδικασία υλοποιείται μέσω των κλήσεων loss.backward() και optimizer.step().

Στην αξιολόγηση, η λογική του forward pass παραμένει ίδια, αλλά η διαδικασία εκτελείται μέσα σε torch.no_grad(), ώστε να μην αποθηκεύονται κλίσεις και να μειώνεται το υπολογιστικό κόστος. Για την παραγωγή τελικών προβλέψεων, στην πολυκλασική περίπτωση επιλέγεται η κλάση με το μέγιστο logit, δηλαδή

$$\hat{y} = \arg\max_c z_c,$$

ενώ στη δυαδική περίπτωση το μοναδικό logit συγκρίνεται με το μηδέν. Επειδή η sigmoid ικανοποιεί $\sigma(0) = 0.5$, η συνθήκη $z > 0$ είναι ισοδύναμη με πιθανότητα μεγαλύτερη από 0.5. Έτσι, η πρόβλεψη δίνεται από

$$\hat{y} = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Οι προβλέψεις και τα πραγματικά labels συλλέγονται ώστε να υπολογιστούν στη συνέχεια οι τελικές μετρικές αξιολόγησης.

Listing 5: Code for train_dataset (training.py)

```python
def train_dataset(_epoch, dataloader, model, loss_function, optimizer):
    model.train()
    running_loss = 0.0

    device = next(model.parameters()).device

    for index, batch in enumerate(dataloader, 1):
        inputs, labels, lengths = batch

        inputs = inputs.to(device)
        labels = labels.to(device)
        lengths = lengths.to(device)

        optimizer.zero_grad()

        pred = model(inputs, lengths)

        if pred.shape[1] == 1:
            loss = loss_function(pred.squeeze(1), labels.float())
        else:
            loss = loss_function(pred, labels.long())

        loss.backward()
        optimizer.step()

        running_loss += loss.data.item()
```

```python

    return running_loss / index
```

Listing 6: Code for eval_dataset (training.py)

```python
def eval_dataset(dataloader, model, loss_function):
    model.eval()
    running_loss = 0.0

    y_pred = []
    y = []

    device = next(model.parameters()).device

    with torch.no_grad():
        for index, batch in enumerate(dataloader, 1):
            inputs, labels, lengths = batch

            inputs = inputs.to(device)
            labels = labels.to(device)
            lengths = lengths.to(device)

            pred = model(inputs, lengths)

            if pred.shape[1] == 1:
                loss = loss_function(pred.squeeze(1), labels.float())
            else:
                loss = loss_function(pred, labels.long())

            if pred.shape[1] == 1:
                y_pred_batch = (pred.squeeze(1) > 0).long()
            else:
                _, y_pred_batch = torch.max(pred, dim=1)

            y_pred.append(y_pred_batch.cpu().numpy())
            y.append(labels.cpu().numpy())
            running_loss += loss.data.item()

    return running_loss / index, (y_pred, y)
```

Ζητούμενο 10 – Αξιολόγηση Baseline Μοντέλου

Πίνακας 2: Αποτελέσματα Baseline μοντέλου

Dataset	Accuracy	F1 (macro)	Recall (macro)
Sentence Polarity (MR)	0.7281	0.7279	0.7281

Σχήμα 1: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Baseline (mean pooling), MR

Ζητούμενο 11 – Κατηγοριοποίηση με Χρήση LLM

Prompts που χρησιμοποιήθηκαν:

Πίνακας 3: Αποτελέσματα ChatGPT

Dataset	Accuracy	F1 (macro)	Recall (macro)
Sentence Polarity	–	–	–
Semeval 2017	–	–	–

Επίδοση μοντέλου:

Ανάλυση αποτελεσμάτων:

Μέρος Β΄: Κύρια Άσκηση

Για τα ερωτήματα 1–8 χρησιμοποιήσαμε το **Sentence Polarity Dataset (MR)** για τα ερωτήματα 1–5 και και το **Semeval 2017 Task4-A** για τα ερωτήματα 6–7.

Ερώτημα 1 – Mean + Max Pooling

1.1 Υλοποίηση: Στο BaselineDNN με pooling='mean_max': mean_rep = embeddings.sum(dim=1) / lengths.float().unsqueeze(1), max_rep = embeddings.max(dim=1).values, representations = torch.cat([mean_rep, max_rep], dim=1). Το dim του linear layer διπλασιάστηκε (dim *= 2).

1.2 Διαφορές από την αρχική αναπαράσταση: Η νέα αναπαράσταση διαφέρει από την αρχική, επειδή η αρχική βασιζόταν αποκλειστικά στον μέσο όρο των embeddings μέσω mean pooling, ενώ τώρα η τελική αναπαράσταση προκύπτει από τη συνένωση (concatenation) του mean pooling και του max pooling. Οι δύο αυτές μορφές pooling παρέχουν διαφορετική αλλά συμπληρωματική πληροφορία, η οποία είναι χρήσιμη για το sentiment analysis των προτάσεων.

Το mean pooling υπολογίζει, για κάθε διάσταση του embedding, τη μέση τιμή της σε όλα τα tokens της πρότασης. Αν θεωρήσουμε ότι κάθε διάσταση του embedding αντιστοιχεί σε κάποια αφηρημένη σημασιολογική ιδιότητα, τότε το mean pooling δίνει μια συνολική εικόνα του νοήματος της πρότασης, σαν ένα είδος κέντρου βάρους. Έτσι αποτυπώνεται το γενικό περιεχόμενο και η μέση κατανομή των χαρακτηριστικών μέσα στην πρόταση. Ωστόσο, αυτή η διαδικασία τείνει να εξομαλύνει πολύ έντονες τοπικές ενδείξεις.

Αντίθετα, το max pooling κρατά, για κάθε διάσταση, τη μεγαλύτερη τιμή που εμφανίζεται σε κάποιο token της πρότασης. Με αυτόν τον τρόπο λειτουργεί σαν μηχανισμός εντοπισμού ισχυρών σημάτων: αν ακόμη και μία μόνο λέξη ενεργοποιεί πολύ έντονα μια συγκεκριμένη διάσταση, αυτή η πληροφορία διατηρείται στην τελική αναπαράσταση. Αυτό είναι ιδιαίτερα χρήσιμο στο sentiment analysis, όπου μία λέξη με έντονη συναισθηματική φόρτιση, θετική ή αρνητική, μπορεί να είναι καθοριστική για την τελική κλάση.

Συνεπώς, η συνένωση των δύο αναπαραστάσεων προσφέρει πλουσιότερη πληροφορία από την αρχική προσέγγιση, επειδή συνδυάζει τόσο τη συνολική, μέση πληροφορία της πρότασης όσο και την παρουσία έντονων χαρακτηριστικών σε κάθε διάσταση.

Πίνακας 4: Αποτελέσματα Ερωτήματος 1 (Mean+Max Pooling)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Mean+Max Pooling	0.7462	0.7454	0.7462

Σχήμα 2: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Mean+Max Pooling, MR

Ερώτημα 2 – LSTM

2.1 Validation set & Early Stopping: Για το validation split χρησιμοποιείται η συνάρτηση torch_train_val_split() στο main.py. Η συνάρτηση αυτή χωρίζει το αρχικό training set σε δύο μέρη: train_loader και val_loader, με default val_size=0.2, άρα χρησιμοποιείται το 20% του training set ως validation set, όπως φαίνεται και στην training.py. Στη συνέχεια εφαρμόζεται early stopping με την κλάση EarlyStopper στο main.py. Σε κάθε εποχή υπολογίζεται το val_loss, και αν αυτό επιδεινώνεται συνεχώς για 5 εποχές, η εκπαίδευση σταματά. Με αυτόν τον τρόπο το μοντέλο δεν εκπαιδεύεται για σταθερό αριθμό εποχών ανεξάρτητα από την απόδοσή του, αλλά σταματά όταν παύει να γενικεύει καλά στο validation set. Αυτό βοηθά στον περιορισμό του overfitting, αλλά και στη μείωση του υπολογιστικού κόστους κατά την εύρεση του βέλτιστου αριθμού εποχών εκπαίδευσης για το δεδομένο dataset.

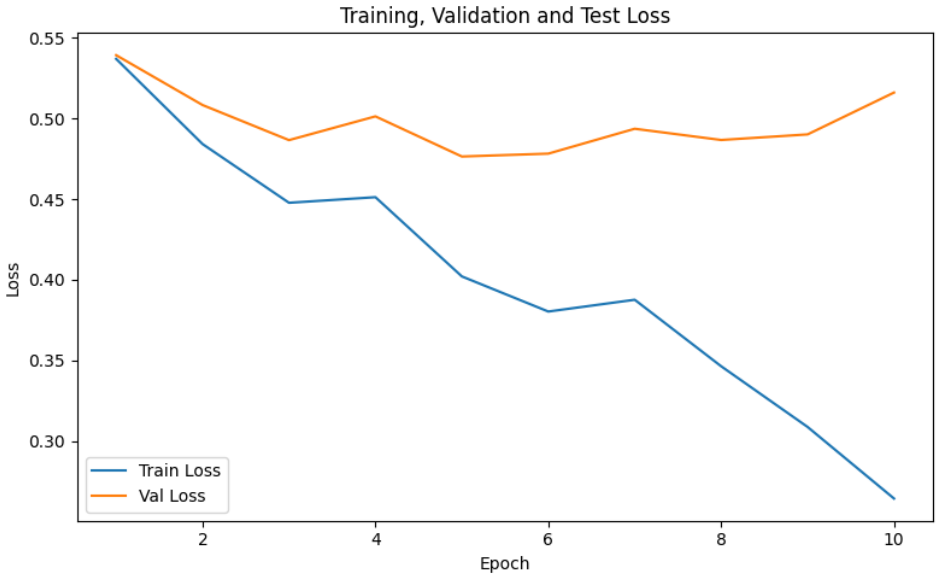
2.2 Χρήση τελευταίας εξόδου LSTM (h_N): Στο LSTM, κάθε λέξη δεν αναπαρίσταται μόνο από το embedding της, αλλά από μια νέα κρυφή κατάσταση h_i που ενσωματώνει και πληροφορία από τα προηγούμενα tokens. Σε αντίθεση με το απλό RNN, το LSTM διαθέτει μηχανισμό μνήμης μέσω του context/cell state c_t και τριών πυλών: forget, input και output gate. Με αυτές το δίκτυο μαθαίνει ποιες πληροφορίες πρέπει να διατηρήσει, ποιες να απορρίψει και ποιες να χρησιμοποιήσει στην τρέχουσα έξοδο. Έτσι μπορεί να μεταφέρει χρήσιμη πληροφορία σε μεγαλύτερες αποστάσεις και να αντιμετωπίζει καλύτερα το πρόβλημα των vanishing gradients. Για την αναπαράσταση της πρότασης χρησιμοποιείται η τελευταία πραγματική έξοδος h_N, δηλαδή η κρυφή κατάσταση που αντιστοιχεί στο τελευταίο έγκυρο token της πρότασης. Αυτό είναι σημαντικό γιατί τα zero-padded timesteps δεν ανήκουν στο πραγματικό κείμενο και δεν πρέπει να επηρεάζουν την αναπαράσταση. Στην υλοποίηση αυτό εξασφαλίζεται με χρήση των πραγματικών μηκών των προτάσεων και του pack_padded_sequence, ώστε το LSTM να επεξεργάζεται μόνο τα αληθινά tokens, και στη συνέχεια επιλέγεται το output στη θέση lengths - 1.

2.3 Αμφίδρομο LSTM (BiLSTM): Στο αμφίδρομο LSTM η πρόταση διαβάζεται και από τις δύο κατευθύνσεις: από αριστερά προς τα δεξιά και από δεξιά προς τα αριστερά. Έτσι η τελική αναπαράσταση κάθε θέσης περιέχει πληροφορία τόσο από το αριστερό όσο και από το δεξί συμφραζόμενο. Αυτό είναι ιδιαίτερα χρήσιμο στη γλώσσα, επειδή η ερμηνεία μιας λέξης συχνά εξαρτάται

όχι μόνο από ό,τι προηγήθηκε αλλά και από ό,τι ακολουθεί. Στην κατηγοριοποίηση προτάσεων, το bidirectional LSTM δίνει συνήθως πιο πλούσια αναπαράσταση του κειμένου από το μονοκατευθυντικό, επειδή συνδυάζει και τα δύο συμφραζόμενα πριν από την τελική ταξινόμηση. Στη δική μας υλοποίηση, με bidirectional=True, το μοντέλο στο dataset MR πέτυχε στο test set τα παρακάτω αποτελέσματα:

Πίνακας 5: Αποτελέσματα Ερωτήματος 2 (LSTM)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
LSTM (unidirectional)	0.7568	0.7546	0.7568



Σχήμα 3: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – LSTM, MR

Ερώτημα 3 – Self-Attention

3.1 Υλοποίηση & Αποτελέσματα (SimpleSelfAttentionModel): Στο SimpleSelfAttentionModel, κάθε λέξη προβάλλεται αρχικά στον χώρο των embeddings και στη συνέχεια, μέσω του μηχανισμού self-attention, παράγεται μια νέα αναπαράσταση για κάθε θέση, η οποία εξαρτάται όχι μόνο από το ίδιο το token αλλά και από τα υπόλοιπα tokens της πρότασης. Έπειτα εφαρμόζεται average-pooling στις τελικές αναπαραστάσεις των λέξεων, ώστε να εξαχθεί μία ενιαία αναπαράσταση για ολόκληρη την πρόταση πριν από το τελευταίο γραμμικό layer ταξινόμησης.

Η επίδοση του μοντέλου στο test set για το dataset MR φαίνεται στον παρακάτω πίνακα. Τα αποτελέσματα δείχνουν ότι τ ο self-attention μοντέλο μπορεί να αξιοποιήσει τις αλληλεπιδράσεις μεταξύ των λέξεων της πρότασης καλύτερα από ένα απλό bag-of-embeddings μοντέλο, αφού η αναπαράσταση κάθε λέξης εμπλουτίζεται με πληροφορία συμφραζομένων πριν από το τελικό pooling.

Πίνακας 6: Αποτελέσματα Ερωτήματος 3 (Self-Attention)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Self-Attention	0.7492	0.7489	0.7492

Σχήμα 4: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Self-Attention, MR

3.2 Queries, Keys, Values και Position Embeddings: Ο μηχανισμός attention μπορεί να ιδωθεί ως ένας σταθμισμένος μέσος όρος διανυσμάτων, όπου τα βάρη εξαρτώνται από το πόσο σχετικό είναι κάθε token με το token που εξετάζουμε κάθε φορά. Στην απλοποιημένη του μορφή, το attention output στη θέση i μπορεί να γραφεί ως

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

όπου τα α_{ij} είναι τα βάρη που καθορίζουν πόσο συμβάλλει η αναπαράσταση x_j στον υπολογισμό της νέας αναπαράστασης a_i.

Στην πλήρη μορφή του self-attention, κάθε είσοδος x_i παίζει τρεις διαφορετικούς ρόλους: ως query, ως key και ως value. Τα queries εκφράζουν τι πληροφορία 'αναζητά' το τρέχον token από τα υπόλοιπα tokens. Τα keys εκφράζουν με ποιον τρόπο κάθε token μπορεί να συγκριθεί με το τρέχον query, ενώ τα values είναι η πραγματική πληροφορία που θα χρησιμοποιηθεί στο τελικό σταθμισμένο άθροισμα.

Στην κλάση attention.Head, για κάθε είσοδο υπολογίζονται τρεις διαφορετικές γραμμικές προβολές:

$$q_i = x_i W_Q, \qquad k_i = x_i W_K, \qquad v_i = x_i W_V$$

Η ομοιότητα μεταξύ του token i και ενός άλλου token j υπολογίζεται από το εσωτερικό γινόμενο του query με το key, κλιμακωμένο με $\sqrt{d_k}$:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Στη συνέχεια, τα scores κανονικοποιούνται με softmax, ώστε να προκύψουν τα βάρη προσοχής:

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

και το τελικό output της κεφαλής προκύπτει ως σταθμισμένο άθροισμα των value vectors:

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} v_j$$

Άρα, το attention output ενός token είναι μια νέα αναπαράσταση που συγκεντρώνει πληροφορία από τα άλλα tokens, δίνοντας μεγαλύτερο βάρος σε όσα θεωρούνται πιο σχετικά.

Τα position embeddings προστίθενται στα token embeddings, επειδή ο μηχανισμός attention από μόνος του δεν περιέχει πληροφορία για τη σειρά ή τη θέση των λέξεων. Χωρίς αυτά, το μοντέλο θα αντιμετώπιζε την πρόταση σαν ένα απλό σύνολο tokens. Με τα position embeddings, κάθε θέση της πρότασης αποκτά δική της διανυσματική πληροφορία, ώστε το μοντέλο να μπορεί να διακρίνει αν μια λέξη εμφανίζεται στην αρχή, στη μέση ή στο τέλος της πρότασης. Έτσι εισάγεται πληροφορία για τη σειρά των tokens, η οποία είναι κρίσιμη για τη σωστή κατανόηση του νοήματος.

Ερώτημα 4 – Multi-Head Attention

Στο ερώτημα αυτό επεκτείνουμε το απλό self-attention σε multi-head attention. Ενώ στο SimpleSelfAttentionModel υπάρχει μία μόνο κεφαλή προσοχής που υπολογίζει μία ενιαία σταθμισμένη σύνοψη των value vectors, στο MultiHeadAttentionModel χρησιμοποιούνται πολλές κεφαλές παράλληλα, καθεμία με δικά της βάρη προβολής για queries, keys και values. Έτσι κάθε κεφαλή μπορεί να εστιάζει σε διαφορετικές σχέσεις ή διαφορετικά πρότυπα μέσα στην πρόταση.

Πιο συγκεκριμένα, για κάθε κεφαλή c, οι είσοδοι προβάλλονται σε ξεχωριστά query, key και value διανύσματα, υπολογίζονται ανεξάρτητα τα βάρη προσοχής και παράγεται ένα επιμέρους head output. Στη συνέχεια, τα αποτελέσματα όλων των κεφαλών συνενώνονται και προβάλλονται ξανά στον αρχικό χώρο αναπαράστασης. Με αυτόν τον τρόπο το μοντέλο αποκτά πλουσιότερη και πιο εκφραστική αναπαράσταση της πρότασης σε σχέση με τη SimpleSelfAttention εκδοχή.

Για κάθε κεφαλή c, οι βασικοί υπολογισμοί περιγράφονται από τις εξισώσεις:

$$q_i^{(c)} = x_i W_Q^{(c)}, \qquad k_j^{(c)} = x_j W_K^{(c)}, \qquad v_j^{(c)} = x_j W_V^{(c)}$$

$$\alpha_{ij}^{(c)} = \text{softmax}\left(\frac{q_i^{(c)} \cdot k_j^{(c)}}{\sqrt{d_k}}\right)$$

$$\text{head}_i^{(c)} = \sum_j \alpha_{ij}^{(c)} v_j^{(c)}$$

και η τελική έξοδος του multi-head attention προκύπτει από τη συνένωση όλων των κεφαλών και την τελική γραμμική προβολή:

$$a_i = \left(\text{head}_i^{(1)} \oplus \text{head}_i^{(2)} \oplus \cdots \oplus \text{head}_i^{(A)}\right) W_O$$

Στη δική μας υλοποίηση, μετά την εφαρμογή του multi-head attention, χρησιμοποιείται average-pooling για την εξαγωγή της τελικής αναπαράστασης της πρότασης πριν από το τελευταίο γραμμικό layer ταξινόμησης. Η επίδοση του μοντέλου στο test set παρουσιάζεται στον αντίστοιχο πίνακα αποτελεσμάτων.

Πίνακας 7: Αποτελέσματα Ερωτήματος 4 (Multi-Head Attention)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
MultiHead-Attention (n_head=4)	0.7477	0.7475	0.7477
MultiHead-Attention (n_head=8)	0.7402	0.7401	0.7402

Σχήμα 5: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Multi-Head Attention, MR

Ερώτημα 5 – Transformer-Encoder

Διαφορά από MultiHeadAttentionModel: Στο ερώτημα αυτό χρησιμοποιούμε έναν Transformer-Encoder για κατηγοριοποίηση συναισθήματος, διατηρώντας το average-pooling για την εξαγωγή της τελικής αναπαράστασης της πρότασης. Όπως και στα προηγούμενα μοντέλα, κάθε πρόταση μετατρέπεται αρχικά σε ακολουθία από token embeddings, στα οποία προστίθενται position embeddings, ώστε το μοντέλο να γνωρίζει τη θέση κάθε token μέσα στην πρόταση. Στη συνέχεια, η ακολουθία περνά από διαδοχικά Transformer blocks, όπου κάθε block αποτελείται από multi-head self-attention, feed-forward network, residual connections και layer normalization. Αν συμβολίσουμε την είσοδο ενός block με x, τότε η βασική του μορφή είναι:

$$x' = x + \text{MultiHeadAttention}(\text{LayerNorm}(x))$$

$$y = x' + \text{FeedForward}(\text{LayerNorm}(x'))$$

Μετά τη διέλευση από όλα τα blocks, εφαρμόζεται average-pooling στις τελικές αναπαραστάσεις των tokens, ώστε να προκύψει μία ενιαία αναπαράσταση πρότασης, η οποία οδηγείται στο τελικό γραμμικό layer ταξινόμησης.

Η βασική διαφορά του TransformerEncoderModel σε σχέση με το MultiHeadAttentionModel είναι ότι το δεύτερο χρησιμοποιεί ουσιαστικά ένα μόνο επίπεδο multi-head attention και ένα feed-forward layer, ενώ ο Transformer-Encoder στοιβάζει πολλά τέτοια blocks διαδοχικά. Άρα ο Transformer-Encoder έχει μεγαλύτερο βάθος και μπορεί να μάθει πιο σύνθετες και πιο αφηρημένες σχέσεις μεταξύ των λέξεων της πρότασης. Επιπλέον, στο τέλος εφαρμόζεται και τελικό layer normalization, το οποίο σταθεροποιεί περισσότερο την αναπαράσταση πριν από την ταξινόμηση.

Default τιμές παραμέτρων στην κλασική αρχιτεκτονική Transformer: Στα πειράματά μας μεταβάλαμε κυρίως τον αριθμό των κεφαλών attention (n_head) και τον αριθμό των encoder layers (n_layer). Στην κλασική αρχιτεκτονική του Transformer οι βασικές default τιμές για τον encoder είναι:

- n_head = 8
- n_layer = 6

- model dimension $d = 512$
- feed-forward dimension $= 2048$
- $d_k = d_v = 64$

Στη δική μας υλοποίηση, επειδή χρησιμοποιούμε προεκπαιδευμένα embeddings διαστάσεων 200, το model dimension καθορίζεται από αυτά και δεν είναι 512. Η επίδοση του μοντέλου στο test set για το dataset MR παρουσιάζεται στον παρακάτω πίνακα αποτελεσμάτων. Παρατηρούμε ότι η καλύτερη επίδοση επιτεύχθηκε με n_head = 4 και n_layer = 6, γεγονός που δείχνει ότι το μεγαλύτερο βάθος βοήθησε το μοντέλο να μάθει πιο εκφραστικές αναπαραστάσεις για την ταξινόμηση συναισθήματος.

Πίνακας 8: Αποτελέσματα Ερωτήματος 5 (Transformer-Encoder)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Transformer-Encoder (n_head=4, n_layer=3)	0.7492	0.7462	0.7492
Transformer-Encoder (n_head=8, n_layer=6)	0.7477	0.7456	0.7477
Transformer-Encoder (n_head=4, n_layer=6)	0.7704	0.7699	0.7704

Σχήμα 6: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Transformer-Encoder, MR

Ερώτημα 6 – Pre-Trained Transformers (Zero-Shot)

Στο ερώτημα αυτό χρησιμοποιήσαμε pre-trained Transformer μοντέλα σε ρύθμιση zero-shot, δηλαδή χωρίς επιπλέον εκπαίδευση πάνω στα δεδομένα της άσκησης. Η υλοποίηση έγινε στο αρχείο transfer_pretrained.py, όπου για κάθε dataset φορτώνεται ένα σύνολο έτοιμων μοντέλων από το HuggingFace μέσω του pipeline("sentiment-analysis"). Για κάθε πρόταση του test set, το μοντέλο επιστρέφει μία ετικέτα συναισθήματος, η οποία μετατρέπεται στην κοινή μορφή των κλάσεων του αντίστοιχου dataset μέσω κατάλληλου label mapping. Στη συνέχεια οι προβλέψεις συγκρίνονται με τις πραγματικές ετικέτες και υπολογίζονται οι μετρικές accuracy, recall (macro) και F1-score (macro).

Για το MR αξιολογήθηκαν πέντε μοντέλα δυαδικής ταξινόμησης: siebert/sentiment-roberta-large-english, distilbert-base-uncased-finetuned-sst-2-english, textattack/bert-base-uncased-SST-2, lvwerra/distilbert-

imdb και aychang/roberta-base-imdb. Για το Semeval2017A αξιολογήθηκαν πέντε μοντέλα τριταξικής ταξινόμησης: cardiffnlp/twitter-roberta-base-sentiment, finiteautomata/bertweet-base-sentiment-analysis, cardiffnlp/twitter-roberta-base-sentiment-latest, cardiffnlp/twitter-xlm-roberta-base-sentiment και Seethal/sentiment_analysis_generic_dataset. Τα αποτελέσματα παρουσιάζονται στους παρακάτω πίνακες.

Από τη σύγκριση των μοντέλων παρατηρούμε ότι στο MR το καλύτερο μοντέλο ήταν το siebert/sentiment-roberta-large-english, με πολύ υψηλή επίδοση, ενώ στο Semeval2017A καλύτερο αποδείχθηκε το cardiffnlp/twitter-roberta-base-sentiment. Το αποτέλεσμα αυτό είναι αναμενόμενο, καθώς τα μοντέλα που είχαν εκπαιδευτεί σε δεδομένα πιο κοντινά στο εκάστοτε domain απέδωσαν καλύτερα: μοντέλα εκπαιδευμένα σε γενικό αγγλικό sentiment λειτούργησαν καλύτερα στο MR, ενώ μοντέλα προσανατολισμένα σε Twitter δεδομένα λειτούργησαν καλύτερα στο Semeval2017A. Συνολικά, οι pre-trained Transformers ξεπέρασαν καθαρά τα μοντέλα που βασίζονται μόνο σε στατικά word embeddings, δείχνοντας το πλεονέκτημα των βαθύτερων προεκπαιδευμένων αναπαραστάσεων.

Πίνακας 9: Αποτελέσματα Pre-Trained Transformers – Sentence Polarity (MR)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
siebert/sentiment-roberta-large-english	0.9260	0.9260	0.9260
textattack/bert-base-uncased-SST-2	0.8988	0.8987	0.8988
distilbert-base-uncased-finetuned-sst-2-english	0.8912	0.8912	0.8912
aychang/roberta-base-imdb	0.8489	0.8485	0.8489
lvwerra/distilbert-imdb	0.8202	0.8195	0.8202

Πίνακας 10: Αποτελέσματα Pre-Trained Transformers – Semeval 2017

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
cardiffnlp/twitter-roberta-base-sentiment	0.7238	0.7222	0.7229
cardiffnlp/twitter-roberta-base-sentiment-latest	0.7205	0.7226	0.7330
finiteautomata/bertweet-base-sentiment-analysis	0.7178	0.7181	0.7302
cardiffnlp/twitter-xlm-roberta-base-sentiment	0.6926	0.6886	0.6929
Seethal/sentiment_analysis_generic_dataset	0.6184	0.6093	0.6181

Σχολιασμός αποτελεσμάτων:

Ερώτημα 7 – Fine-Tuning Pre-Trained Transformers

Πίνακας 11: Αποτελέσματα Fine-Tuned Transformers – Sentence Polarity

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**

Πίνακας 12: Αποτελέσματα Fine-Tuned Transformers – Semeval 2017

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**

Σχολιασμός αποτελεσμάτων:

Ερώτημα 8 – Διάλογος με ChatGPT για GPT Κώδικα

Παραδείγματα διαλόγου:

Σχολιασμός απόδοσης:

Συγκεντρωτικά Αποτελέσματα

Πίνακας 13: Σύγκριση όλων των μοντέλων – **Sentence Polarity (MR)** (εκτός Pre-Trained Semeval)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Baseline DNN (mean pooling)	0.7281	0.7279	0.7281
Mean + Max Pooling	0.7462	0.7454	0.7462
LSTM	0.7568	0.7546	0.7568
Self-Attention	0.7492	0.7489	0.7492
Multi-Head Attention (n_head=4)	0.7477	0.7475	0.7477
Transformer-Encoder (n_head=4, n_layer=6)	0.7704	0.7699	0.7704
Pre-Trained best (siebert, MR)	0.9260	0.9260	0.9260
Pre-Trained best (cardiffnlp, Semeval)	0.7238	0.7222	0.7229
Fine-Tuned (best)	–	–	–